Minimal Checkout with Fungies — Step-by-Step

A practical guide to wiring up the smallest possible working checkout where:

- Customer details (email, country) are **pre-filled**.
- You pass `workspace_id` and `user_id` as **custom fields**.
- Those custom fields come back to you in the **webhook**.
- The checkout **closes / redirects on success**.

This is the integration recipe the official Fungies docs describe, cross-checked against the real behaviour of the platform. The 3 most common reasons your first attempt won't work are all spelled out in the Troubleshooting section at the end.

---

1. Decide which checkout flavor to use

Fungies gives you 3 ways to render the same hosted checkout. Pick **one** and stick with it.

Mode	What the user sees	Best for	Effort
Hosted	Browser navigates to a Fungies-hosted page	Email links, simple buttons, marketing campaigns	Trivial — just a URL
Overlay	A full-screen modal pops over your site	SaaS dashboards, "buy" buttons, keeping the user on your domain	Add 1 SDK script + 1 button
Embed	An iframe is rendered inside an element you control	Dedicated `/checkout` pages	Add 1 SDK script + 1 container div

For a "show a popup with the customer's email pre-filled, close on success" UX, **use Overlay**. The rest of this guide focuses on Overlay, then shows the 2-line variants for Hosted and Embed.

---

2. Prerequisites — what to set up in the Dashboard FIRST

90% of "it doesn't work" issues come from skipping this section. Do these in order.

2.1 Create the product and an offer

Dashboard → Products → Game Assets → Add product. Create a `OneTimePayment` or `Subscription` product, then create at least one **offer** under it (price + currency). Note the offer ID — that's what goes in the checkout URL.

(If you're building programmatically, use `POST /v0/products/create` + `POST /v0/offers/create`. See `creating-products-and-offers.pdf` in this folder.)

2.2 Define your custom fields at PROJECT level

This is the single most common gotcha. **You cannot just append `?workspace_id=foo` to the URL and expect it in the webhook.** Fungies only echoes back fields that are declared at project level.

Dashboard → Products → Game Assets → click your **project** → scroll to **Custom Fields** → **Add field**, twice:

Label	Field key (what your code sends)	Type	Required	Regex (optional)
Workspace ID	`workspace_id`	Text	Yes (default)	`^[A-Za-z0-9_-]{1,64}$`
User ID	`user_id`	Text	Yes (default)	`^[A-Za-z0-9_-]{1,64}$`

Save the project. From this moment on, **all custom fields are required by default** — checkout will refuse to load unless every defined field is supplied. So pass them all from the very first integration test.

2.3 Configure your webhook endpoint

Dashboard → Developers → Webhooks → Add endpoint.

- URL: `https://your-app.example.com/api/fungies/webhook`
- Events to subscribe (minimum): `order.paid`, `payment_success`, `payment_failed`, `payment_refunded`, `subscription_created`, `subscription_interval`, `subscription_cancelled`.
- Copy the **signing secret** — you'll verify the `x-fngs-signature` HMAC-SHA256 header against it on every incoming request.

2.4 Configure the post-purchase Instant Redirect URL

Dashboard → Settings → Store → **Checkout** tab.

- Set **Instant Redirect URL** to wherever you want the customer to land after success, e.g. `https://your-app.example.com/welcome`.
- Add the system parameters `fngs-order-id` and `fngs-user-email`. Fungies will append them to the redirect, so your page can use them to recover state.

This is what makes the overlay close itself and the customer return to your app on success. **Without this set, the overlay sits there showing the success screen and never closes.**

2.5 Publish the store

Dashboard → Store → Publish. The `/checkout/<offer_id>` URLs only resolve once the store is in the published state.

---

3. The exact URL / parameter shape (this is where most bugs live)

Concept	Hosted query param	Overlay/Embed (SDK option)	Overlay/Embed (HTML attribute)
Email prefill	`fngs-customer-email`	`billingData.email`	`data-fungies-billing-email`
First name	`fngs-customer-first-name`	`billingData.firstName`	`data-fungies-billing-first-name`
Last name	`fngs-customer-last-name`	`billingData.lastName`	`data-fungies-billing-last-name`
Country (ISO-2)	`fngs-customer-country`	`billingData.country`	`data-fungies-billing-country`
State	`fngs-customer-state`	`billingData.state`	`data-fungies-billing-state`
City	`fngs-customer-city`	`billingData.city`	`data-fungies-billing-city`
ZIP	`fngs-customer-zip-code`	`billingData.zipCode`	`data-fungies-billing-zip-code`
Quantity	`fngs-quantity`	`quantity`	`data-fungies-quantity`
Discount code	`fngs-discount-code`	`discountCode`	`data-fungies-discount-code`
Custom field `user_id`	`user_id` (bare key)	`customFields.user_id`	inside `data-fungies-custom-fields` JSON

The key point: billing-data params have a `fngs-customer-` / *`billingData.`* / `data-fungies-billing-*` prefix. Custom fields are passed by their **bare key name** — no prefix at all. Mixing these up is the #1 reason your prefill doesn't work.

> The legacy `customerEmail` SDK property and `data-fungies-customer-email` attribute are still accepted but deprecated. Don't use them in new code.

> Note: `fngs-user-email` (with **user**, not **customer**) is **not** a prefill parameter. It is the system param that Fungies *appends* to your Instant Redirect URL after purchase. Don't pass it on the way in.

---

4. Overlay checkout — the minimal HTML

Copy this into a static `.html` file, replace the three placeholders, open it in a browser, click the button:

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <title>Buy Pro Plan</title>
</head>
<body>
  <button
    id="buy-button"
    data-fungies-checkout-url="https://YOUR-STORE.app.fungies.io/checkout/YOUR-OFFER-ID"
    data-fungies-mode="overlay"
    data-fungies-billing-email="alice@example.com"
    data-fungies-billing-country="US"
    data-fungies-custom-fields='{"workspace_id":"ws_42","user_id":"usr_abc123"}'>
    Buy Pro Plan
  </button>

  <script
    src="https://cdn.jsdelivr.net/npm/@fungies/fungies-js@latest"
    defer
    data-auto-init>
  </script>
</body>
</html>
```

That's the whole integration. What this does:

1. The Fungies JS SDK loads from CDN and `data-auto-init` makes it scan the DOM for buttons.
2. Click handler opens the checkout in a full-screen overlay.
3. Email and country are pre-filled in the checkout form.
4. `workspace_id` and `user_id` are attached to the order and will appear in every webhook event for that order.
5. On payment success Fungies redirects to your Instant Redirect URL (configured in step 2.4) and the overlay closes itself.

---

5. The same thing in plain JavaScript (no data attributes)

Useful when the values are dynamic (e.g. from your auth state in a SPA):

```
<button id="buy-button">Buy Pro Plan</button>

<script src="https://cdn.jsdelivr.net/npm/@fungies/fungies-js@latest"></script>
<script>
  Fungies.Initialize();

  document.getElementById("buy-button").addEventListener("click", () => {
    Fungies.Checkout.open({
      checkoutUrl: "https://YOUR-STORE.app.fungies.io/checkout/YOUR-OFFER-ID",
      settings: { mode: "overlay" },
      billingData: {
        email:   currentUser.email,
        country: currentUser.countryCode  // "US", "PL", "DE", ...
      },
      customFields: {
        workspace_id: currentUser.workspaceId,
        user_id:      currentUser.id
      }
    });
  });
</script>
```

To close the overlay programmatically (e.g. on a custom cancel button somewhere):

```
Fungies.Checkout.close();
```

---

6. Hosted checkout (no JS) — just a URL

If you don't want to load the SDK at all (e.g. you're sending a magic link in an email), use the bare URL:

```
https://YOUR-STORE.app.fungies.io/checkout/YOUR-OFFER-ID
  ?fngs-customer-email=alice%40example.com
  &fngs-customer-country=US
  &workspace_id=ws_42
  &user_id=usr_abc123
```

URL-encode the values (`@` → `%2540` ... actually `%40` ; `space` → `%20`). The custom-field keys go in **bare**, the billing prefill keys use the `fngs-customer-*` prefix.

On success Fungies redirects the customer to your Instant Redirect URL.

---

7. Embedded checkout (iframe inside your page)

Same SDK, different `mode` :

```
<div id="checkout-container" style="min-height: 700px;"></div>

<script src="https://cdn.jsdelivr.net/npm/@fungies/fungies-js@latest"></script>
<script>
  Fungies.Initialize();

  Fungies.Checkout.open({
    checkoutUrl: "https://YOUR-STORE.app.fungies.io/checkout/YOUR-OFFER-ID",
    settings: {
      mode: "embed",
      frameTarget: "checkout-container"
    },
    billingData: { email: "alice@example.com", country: "US" },
    customFields: { workspace_id: "ws_42", user_id: "usr_abc123" }
  });
</script>
```

Embed mode is fire-and-forget — the iframe stays mounted until purchase or until you navigate away. On success the iframe redirects internally to your Instant Redirect URL inside its own frame, so it's usually cleaner to break out via `window.top.location.href = '/welcome'` from your own server's redirect page, or just use overlay mode instead.

---

8. What the webhook looks like with custom fields

Once payment succeeds, Fungies POSTs to your endpoint. The `customFields` you defined and passed come back under `data.customFields` (and per-line-item under `data.items[].customFields`):

```
{
  "object": "event",
  "id": "evt_6f97c060",
  "type": "order.paid",
  "createdAt": 1716557400000,
  "data": {
    "object": "order",
    "id": "ord_xyz789",
    "customFields": {
      "workspace_id": "ws_42",
      "user_id":      "usr_abc123"
    },
    "items": [
      {
        "id": "a0bf335d",
        "name": "Pro Plan",
        "quantity": "1",
        "customFields": {
          "workspace_id": "ws_42",
          "user_id":      "usr_abc123"
        }
      }
    ]
  }
}
```

Your handler should:

1. Verify the HMAC-SHA256 `x-fngs-signature` header against your webhook secret (with a timing-safe comparison).
2. Look up the user using `data.customFields.user_id` and `data.customFields.workspace_id`.
3. Be **idempotent** — key on `event.id` and ignore retries (Fungies guarantees at-least-once delivery with up to 5 retries).
4. Return a `2xx` within 30 seconds.

---

9. Complete copy-paste example (production-ready skeleton)

This is a single file that demonstrates everything: prefill, custom fields, and the post-purchase redirect handoff to your own page.

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <title>Subscribe</title>
  <style>
    .pay-btn {
      background: #6035EC; color: white; border: 0;
      padding: 12px 24px; border-radius: 8px;
      font-size: 16px; cursor: pointer;
    }
    .pay-btn:hover { background: #4c2abd; }
  </style>
</head>
<body>
  <h1>Pro Plan &mdash; $19/month</h1>
  <button id="buy" class="pay-btn">Subscribe</button>

  <script src="https://cdn.jsdelivr.net/npm/@fungies/fungies-js@latest"></script>
  <script>
    // 1. Whatever your auth gives you:
    const currentUser = {
      id:          "usr_abc123",
      email:       "alice@example.com",
      countryCode: "US",
      workspaceId: "ws_42"
    };

    Fungies.Initialize();

    document.getElementById("buy").addEventListener("click", () => {
      Fungies.Checkout.open({
        checkoutUrl: "https://YOUR-STORE.app.fungies.io/checkout/YOUR-OFFER-ID",
        settings: { mode: "overlay" },
        billingData: {
          email:   currentUser.email,
          country: currentUser.countryCode
        },
        customFields: {
          workspace_id: currentUser.workspaceId,
          user_id:      currentUser.id
        }
      });
    });
  </script>
</body>
</html>
```

And the matching minimal webhook handler (Node/Express sketch):

```
import express from "express";
import crypto from "node:crypto";

const app = express();
app.use(express.json({
  verify: (req, _res, buf) => { req.rawBody = buf; }
}));

const SECRET = process.env.FUNGIES_WEBHOOK_SECRET;
const seen = new Set(); // replace with Redis / DB in production

app.post("/api/fungies/webhook", (req, res) => {
  const sig = req.header("x-fngs-signature") || "";
  const expected = "sha256_" + crypto
    .createHmac("sha256", SECRET)
    .update(req.rawBody)
    .digest("hex");

  if (sig.length !== expected.length ||
      !crypto.timingSafeEqual(Buffer.from(sig), Buffer.from(expected))) {
    return res.status(401).end();
  }

  const evt = req.body;
  if (seen.has(evt.id)) return res.status(200).end(); // idempotency
  seen.add(evt.id);

  if (evt.type === "order.paid" || evt.type === "payment_success") {
    const { workspace_id, user_id } = evt.data.customFields || {};
    // grant access in your DB...
    console.log("paid", { workspace_id, user_id, order: evt.data.id });
  }

  res.status(200).end();
});

app.listen(3000);
```

---

10. Troubleshooting — exact symptoms from your question

"Email isn't pre-filling"

Almost certainly the parameter name is wrong. Audit your request against the table in section 3. Most common mistakes:

You wrote	Should be
`?fngs-user-email=...`	`?fngs-customer-email=...`
`?email=...` (bare)	`?fngs-customer-email=...`
`data-fungies-email="..."`	`data-fungies-billing-email="..."`
`customerEmail: "..."` (SDK)	`billingData: { email: "..." }`

Secondary cause: billing fields are only **shown** in checkout when "Collect billing information" is enabled in the dashboard checkout settings (or your Stripe account is India-based, where billing is regulatory-required). If they're not shown, the prefill still attaches to the order but isn't visible — check the order in the dashboard, not the visible form.

"Custom fields are missing from my webhook payload"

Three possible causes, in order of frequency:

1. **You didn't define them in the Dashboard.** Custom fields only flow through to the webhook if they exist as project-level fields. See section 2.2.
2. **You're passing them with a `fngs-` prefix.** Custom fields use **bare keys** (`?user_id=...`), not `?fngs-user_id=...`.
3. **Validation is failing silently.** If you set a regex on the custom field and the value you pass doesn't match, checkout refuses to load — the customer never sees the form. Inspect the browser console for `400`s when the checkout opens.

"The overlay/popup doesn't close on successful subscription"

You're missing the **Instant Redirect URL** in Dashboard → Settings → Store → Checkout tab. Without it the success screen sits inside the overlay forever. With it set, Fungies navigates the overlay frame to your redirect URL on success, which effectively closes the popup and drops the customer on your page.

If you want to also handle "user closed the modal without paying" in your own UI, listen for `Fungies.Checkout.close()` being called — or just leave it; the overlay backdrop has its own close affordance.

"Hosted, overlay, and embedded all behave differently for me"

They share the same checkout backend, so the difference is purely the **wrapper**. Make sure:

- Hosted → Use the bare offer URL with `?fngs-customer-*` query params + bare custom-field keys.
- Overlay → Use the SDK with `mode: "overlay"` and the `billingData` / `customFields` objects.
- Embed → Same as overlay but `mode: "embed"` and a `frameTarget` element.

Don't mix paradigms (e.g. don't pass `data-fungies-billing-email` AND `?fngs-customer-email=` — pick the one for your mode).

"Price isn't pre-filled"

Price is **always** controlled by the **offer**, not by URL parameters. Whatever you set on the offer in the Dashboard (or via `POST /v0/offers/create`) is what the customer sees. If you want a different price for the same product, create a second offer with that price and link to it. There's no way to override price client-side, by design (otherwise customers could rewrite it in DevTools).

To pass quantity, use `?fngs-quantity=3` (hosted) or `quantity: 3` (SDK).

---

11. Checklist before going live

- [ ] Custom fields `workspace_id` and `user_id` exist as project-level fields in the Dashboard.
- [ ] Webhook endpoint registered and signing-secret stored in your server's env.
- [ ] Webhook handler verifies `x-fngs-signature` (HMAC-SHA256, timing-safe).
- [ ] Webhook handler is idempotent on `event.id`.
- [ ] Instant Redirect URL configured in Dashboard → Settings → Store → Checkout.
- [ ] Store is published.
- [ ] You're using **production keys + production store URL** (no `stage.fungies.net` left in code).
- [ ] One end-to-end test purchase performed with a Stripe test card on staging first.

---

12. References

- Checkout Elements overview: https://docs.fungies.io/developers/checkout-elements/overview
- Billing Data Prefill: https://docs.fungies.io/developers/checkout-elements/billing-data
- JavaScript SDK: https://docs.fungies.io/developers/checkout-elements/sdk
- HTML Data Attributes: https://docs.fungies.io/developers/checkout-elements/html-attributes
- Custom Fields Overview: https://docs.fungies.io/developers/customer-data/overview
- Set Up Custom Fields: https://docs.fungies.io/developers/customer-data/setup
- Validate Customer Data: https://docs.fungies.io/developers/customer-data/validate
- Webhooks Overview: https://docs.fungies.io/developers/webhooks/overview
- Set Up Webhooks: https://docs.fungies.io/developers/webhooks/setup

---